Turning Gold into Damage: A Simple Linear Regression for Jinx in Ranked League of Legends*

MATH 261A Project 1

Ryan Rajaie

September 25, 2026

In League of Legends, players earn gold during a match and spend it on items that increase their character's damage. Using 2,795 ranked solo/duo games of the champion Jinx collected through the Riot Games API, I fit a simple linear regression of damage dealt to enemy champions per minute on gold earned per minute. Each additional 100 gold per minute is associated with about 315 more damage per minute, and gold income explains about 65% of the variation in damage output. Residual diagnostics show that the spread of damage grows with gold income, but the relationship is strong enough that this does not change the conclusion. The results give a simple baseline for how efficiently a typical Jinx player converts resources into damage, though they describe an association rather than a causal effect of gold.

1 Introduction

In League of Legends (Riot Games 2009), a team strategy game made by Riot Games, two teams of five players compete to destroy the other team's base. During a match, each player earns gold by killing enemy minions, monsters, and champions, and spends it on items that make their champion stronger. Players and coaches often talk about gold as the currency of the game: a player who is "ahead" has more gold, more items, and therefore more ability to damage opponents. Jinx is a marksman champion (Riot Games 2026a) who usually plays in the bottom lane and whose damage depends heavily on purchased items, which makes her a natural case for asking how strongly gold income and damage output are related.

*Project repository available at: https://github.com/R-Rajaie/lol-scraper

Most published statistical work on League of Legends studies professional matches and models team-level outcomes such as winning. For example, Novak et al. (2020) modelled performance at the 2018 World Championship using team statistics. Professional games are a small and unusual sample, and team-level models do not describe how one player's resources relate to that player's output. Ranked solo/duo games cover millions of ordinary players, and the Riot Games API (Riot Games 2026b) makes per-player statistics from those games available. I found no simple, published estimate of how much damage a typical ranked player on a single champion produces per unit of gold.

In this project I fit a simple linear regression of Jinx's damage to champions per minute on her gold earned per minute, using 2,795 ranked games from three server regions and all rank tiers. I restrict the analysis to one champion so that the slope is not driven by differences between champions (for example, support champions earn little gold and deal little damage by design). The estimated slope is about 3.15 damage per gold: a Jinx player earning 100 more gold per minute deals about 315 more damage per minute on average. The relationship is strong but not perfectly linear, and the variability of damage grows with gold. The estimate gives players and analysts a baseline for judging whether a given game's damage is high or low for the gold earned.

Section 2 introduces the data and the cleaning steps. Section 3 describes the model, its assumptions, and how I checked them. Section 4 presents the fitted model and diagnostics, and Section 5 discusses what the results mean, their limitations, and possible extensions.

2 Data

2.1 Source and collection

The data come from the Riot Games Match-V5 API (Riot Games 2026b), which returns a detailed record of each completed match, including end-of-game statistics for all ten participants. Riot provides this data to registered developers under its API policies (Riot Games 2026c). I collected matches with a Python script in the project repository. The script samples ranked players from Riot's league ladders in three server regions (North America, Europe West, and Korea) and four rank groups: low (Iron to Silver), mid (Gold to Platinum), high (Emerald to Diamond), and elite (Master to Challenger). Within each group, players are drawn at random across all divisions. For each sampled player, the script downloads their 20 most recent ranked solo/duo matches. In total, 21,860 matches were collected.

Because matchmaking pairs players of similar skill, I label each match with the rank group of the sampled player it was downloaded for, which approximates the skill level of the whole match. Most games (96%) were played on patches from the 2026 season (version 16), so the data mostly reflect one version of the game.

2.2 Observational units and cleaning

The observational unit is one Jinx player in one match. A champion can be picked by at most one player per ranked match, so each match contributes at most one row. All 2,882 matches containing Jinx are ranked solo/duo games (Riot queue 420). From these, I removed:

- 74 games that ended in a "remake" (an early vote to cancel the match, usually because a player disconnected) or lasted under five minutes. Their statistics are near zero and do not reflect a played game.
- 13 games where Jinx was not played in the bottom lane. Jinx in another role faces different opponents and earns gold differently, and these games are too few to study separately.

No remaining rows had missing or negative values for the variables used. The final dataset has 2,795 games from 2,508 distinct players; only 424 games come from players who appear more than once.

2.3 Variables

- **Gold per minute** (`gpm`, the predictor) is `goldEarned`, the total gold the player earned during the match, divided by the match length in minutes. Gold earned includes gold from minions, monsters, kills, assists, objectives, and passive income over time.
- **Damage per minute** (`dpm`, the response) is `totalDamageDealtToChampions`, the total damage the player dealt to enemy champions, divided by match length in minutes. It excludes damage to minions, monsters, and structures.
- **Match length** (`gameDuration`) is recorded by Riot in seconds.

I use per-minute rates instead of totals because both gold and damage accumulate over time. With totals, a long game would have high values of both variables even for a poor performance, and the regression would mostly measure game length. Both variables are recorded automatically by the game server, so measurement error from data entry is not a concern. However, `totalDamageDealtToChampions` counts damage dealt, not damage that led to kills, so it is an imperfect measure of useful output.

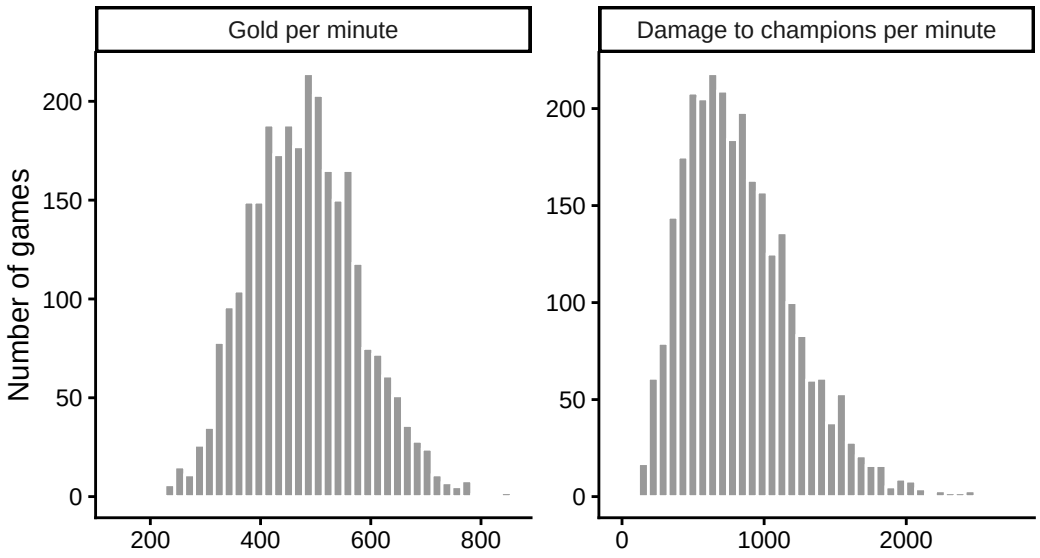


Figure 1: Distributions of gold earned per minute (left) and damage dealt to champions per minute (right) for 2,795 ranked Jinx games. Gold per minute is roughly symmetric around 480, while damage per minute is right-skewed, with a long tail of games above 1,500.

Figure 1 shows the distribution of each variable. Gold per minute has a mean of 479 and a standard deviation of 99. Damage per minute has a mean of 829 and a standard deviation of 387, and it is right-skewed: a few games have very high damage, but damage cannot fall below zero.

Table 1: Summary of the cleaned Jinx games by rank group of the match, with average gold per minute, damage per minute, game length and win rate. Rank groups are low (Iron to Silver), mid (Gold to Platinum), high (Emerald to Diamond), and elite (Master to Challenger). Average gold and damage per minute rise with rank, except that elite games have slightly lower damage than high games.

Rank	Games	Gold/min	Damage/min	Length (min)	Win %
low	715	440	766	29.9	49.4
mid	765	479	831	29.4	51.8
high	776	496	875	28.1	53.0
elite	539	508	844	26.3	54.0

Table 1 breaks the sample down by rank group. The groups are close to evenly represented, and average gold per minute increases with rank, so rank is one source of variation in the predictor.

Riot's API also provides minute-by-minute match timelines, and third-party sites such as League of Graphs (League of Graphs 2026) publish aggregate champion statistics. I use end-of-game statistics because they are complete for every match and match the question about a player's overall game.

3 Methods

3.1 Model

I fit the simple linear regression model (Kutner et al. 2005, Ch. 1)

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i, \qquad i = 1, \dots, n,$$

where $n = 2{,}795$ is the number of games, Y_i is Jinx's damage to champions per minute in game i, X_i is her gold earned per minute in game i, treated as a known constant, and ε_i is a random error with mean zero and finite variance σ^2, with errors independent across games. The error represents everything else that affects damage in that game, such as the player's skill, item choices, and the opposing team. Under this model, the mean damage per minute in games with gold per minute X_i is $E(Y_i) = \beta_0 + \beta_1 X_i$, and the variance around that mean is $\mathrm{Var}(Y_i) = \sigma^2$.

The slope β_1 is the change in mean damage per minute for each one-unit increase in gold per minute. The intercept β_0 is the mean damage per minute when gold per minute is zero. No real game has zero gold income (the minimum in the data is 146), so β_0 positions the line and has no practical meaning on its own.

I chose gold as the predictor and damage as the response because gold buys items, and items increase damage. The reverse also happens, because damage leads to kills, which pay gold. The model therefore describes an association and cannot estimate the causal effect of giving a player more gold.

3.2 Estimation and inference

I estimate β_0 and β_1 by ordinary least squares (OLS), where the estimates b_0 and b_1 are the values of β_0 and β_1 that minimize $Q = \sum_{i=1}^{n}(Y_i - \beta_0 - \beta_1 X_i)^2$. The fitted value for game i is $\widehat{Y}_i = b_0 + b_1 X_i$, and the residual is $e_i = Y_i - \widehat{Y}_i$. I estimate σ^2 with $s^2 = \mathrm{MSE} = \mathrm{SSE}/(n-2)$, where $\mathrm{SSE} = \sum_{i=1}^{n} e_i^2$, and call s the residual standard error. The coefficient of determination $R^2 = 1 - \mathrm{SSE}/\mathrm{SSTO}$, where $\mathrm{SSTO} = \sum_{i=1}^{n}(Y_i - \overline{Y})^2$, is the proportion of the variation in damage per minute explained by the fitted line.

I test $H_0 : \beta_1 = 0$ against $H_A : \beta_1 \neq 0$ with a type I error rate of $\alpha = 0.05$, using the test statistic $t^* = b_1 / \widehat{\text{se}}(b_1)$, where $\widehat{\text{se}}(b_1) = s / \sqrt{\sum_{i=1}^{n} (X_i - \overline{X})^2}$. Under the normal error regression model and H_0, t^* follows a $t(n-2)$ distribution, and I reject H_0 if $|t^*| > t_{0.975,\, n-2}$. A 95% confidence interval for β_1 is $b_1 \pm t_{0.975,\, n-2} \times \widehat{\text{se}}(b_1)$.

Following Gelman, Hill, and Vehtari (2020, Ch. 11), I consider the assumptions of the analysis in decreasing order of importance:

1. **Validity:** the variables measure what the question is about. Damage to champions counts all damage dealt, not only damage that decided a fight.
2. **Representativeness:** the sample resembles all ranked Jinx games. Higher ranks are over-sampled, which matters only if the relationship differs by rank.
3. **Linearity:** $E(Y_i) = \beta_0 + \beta_1 X_i$.
4. **Independence:** the errors ε_i are independent across games.
5. **Equal variance:** the errors have the same variance σ^2 at every value of X_i.
6. **Normality:** the errors are normally distributed.

Following Kutner et al. (2005, Ch. 3), I check linearity and equal variance with a plot of residuals against fitted values, and normality with a normal quantile-quantile (Q-Q) plot. I check representativeness by refitting the model within each rank group. Independence is supported by the design: each match contributes one Jinx player, and few players appear more than once. Unequal variance leaves b_1 unbiased but makes its standard error approximate, while non-normality matters little with thousands of games.

A single predictor leaves out other influences on damage, such as rank. Adding rank as a second predictor could move the slope in either direction, and modelling $\log Y_i$ could make the variance more equal. I return to these extensions in Section 5.

3.3 Software

I performed the analysis in R (R Core Team 2024). The raw JSON match files were read with the `jsonlite` package, and the data were processed and plotted with the `tidyverse` (Wickham et al. 2019). I fit the regression model with R's `lm()` function. The project report was written in Quarto.

4 Results

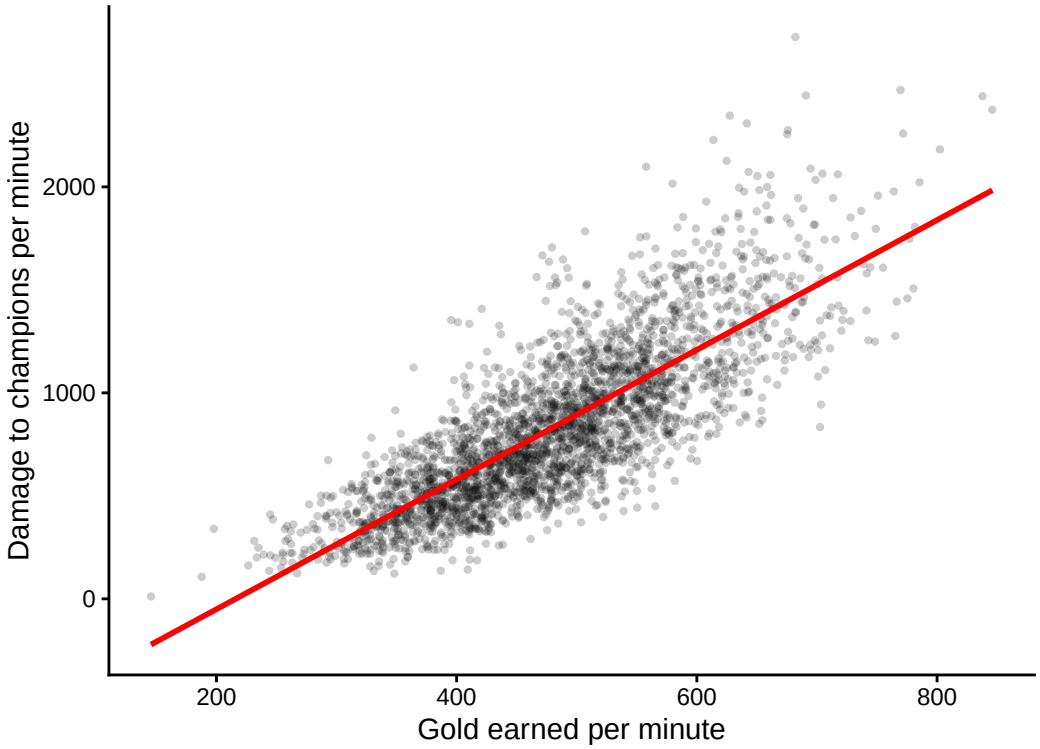


Figure 2: Damage to champions per minute against gold earned per minute for 2,795 ranked Jinx games, with the fitted least-squares line (red). Games with more gold income have higher damage output, and the points spread out more at higher gold income.

Figure 2 shows a strong positive linear relationship between gold and damage per minute; the sample correlation is 0.80. The fitted regression line is

$$\widehat{Y} = -680.2 + 3.149X.$$

Table 2: Estimated coefficients for the regression of damage to champions per minute on gold earned per minute (n = 2,795). The 95% confidence interval for the slope is far from zero.

Term	Estimate	Std. error	95% CI
Intercept (b_0)	-680.2	21.560	(-722.52, -637.97)
Gold per minute (b_1)	3.149	0.044	(3.06, 3.24)

The estimated slope is $b_1 = 3.149$ (Table 2). For each additional gold earned per minute, the expected damage to champions per minute increases by about 3.15. On a more useful scale, a Jinx player who earns 100 more gold per minute, roughly one standard deviation, deals about 315 more damage per minute on average. The estimated intercept is b_0 = -680.2. A negative damage value is impossible, which confirms that the intercept only positions the line and should not be interpreted directly.

The t-statistic for $H_0 : \beta_1 = 0$ is $t^* = 71.5$ with 2,793 degrees of freedom, giving a p-value below 10^{-15}. Because $|t^*|$ is far above the critical value $t_{0.975,\, n-2} = 1.96$, and the p-value is far below $\alpha = 0.05$, I reject the null hypothesis: gold and damage per minute are linearly associated. The 95% confidence interval for β_1 is (3.06, 3.24). If the sampling and analysis were repeated many times, 95% of intervals built this way would contain the true slope; the interval therefore gives a range of plausible values for the increase in mean damage per minute for each extra gold per minute. The coefficient of determination is $R^2 = 0.647$, so gold per minute accounts for about 65% of the variation in damage per minute. The residual standard error is $s = 230$ damage per minute, so a typical game is about 230 damage per minute above or below the line.

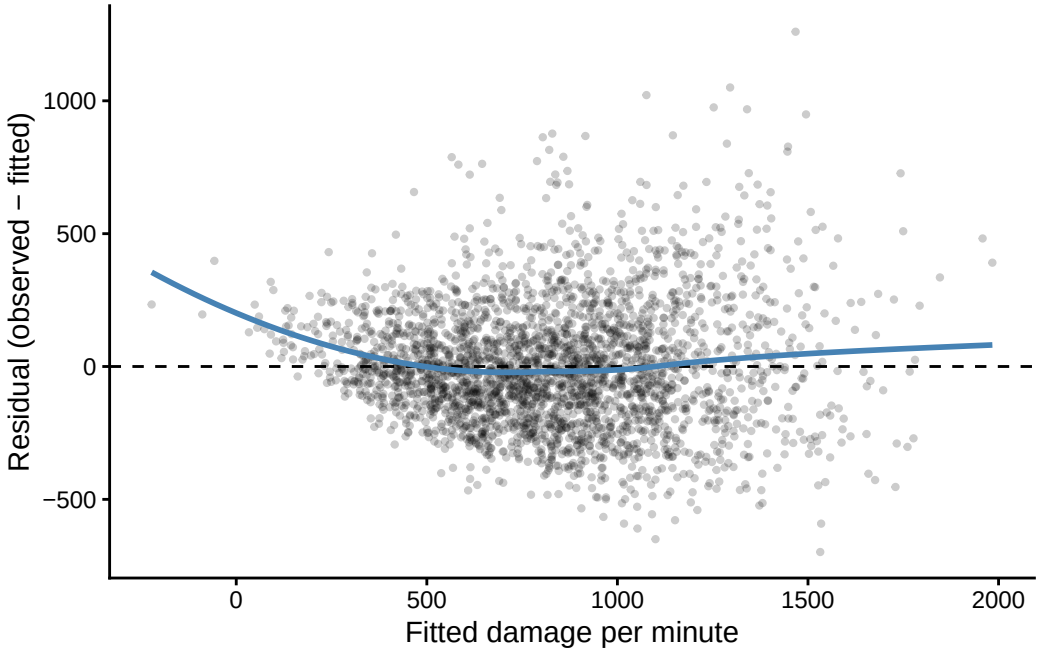


Figure 3: Residuals against fitted values for the regression of damage per minute on gold per minute. The dashed line marks zero and the blue curve is a smoothed trend. The vertical spread of the residuals increases from left to right, showing that the error variance grows with predicted damage. The curve bends slightly upward at both ends, showing mild curvature.

Figure 3 checks the linearity and equal-variance assumptions. The residuals are centred near zero across most of the fitted range, with a mild upward bend at both ends, so a straight line describes the trend well. Their spread increases, however: the residual standard deviation is 172 in the lowest third of fitted values, 224 in the middle third, and 281 in the highest third, so the equal-variance assumption is not met.

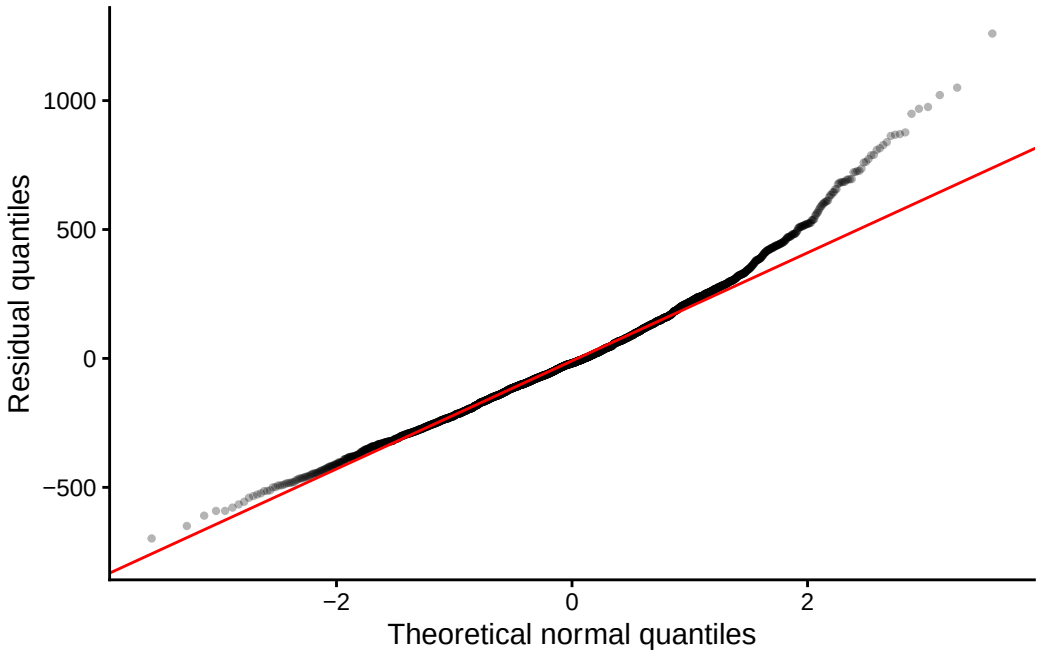


Figure 4: Normal Q-Q plot of the regression residuals. Points on the line would indicate normally distributed errors. The upper tail rises above the line, showing that the residuals are right-skewed: some games have much more damage than predicted, while very large shortfalls are less common.

Figure 4 shows that the residuals are right-skewed, so the normality assumption is also not met. With 2,795 observations, the sampling distribution of b_1 is still close to normal by the central limit theorem, so this departure has little effect on the test. The unequal variance matters more, because the reported standard error assumes a constant σ^2 and is therefore only approximate. It would have to be more than 36 times too small for the slope to stop being significant, so the conclusion that gold and damage are positively associated does not depend on this assumption.

To check representativeness, I fit the same model separately within each rank group. The slopes range from 3.03 (elite) to 3.50 (mid), all close to the pooled estimate, so over-representing higher ranks has little effect on the overall slope.

5 Discussion

This analysis asked how strongly a Jinx player's gold income is related to the damage she deals to enemy champions. In ranked solo/duo games across all skill levels, the relationship is strong and positive: 100 extra gold per minute goes with about 315 extra damage per minute, and gold income explains about 65% of the variation in damage. For players and coaches, the fitted line gives a simple benchmark for average output at a given gold income. A game far below the line suggests the player earned gold but did not turn it into damage, for example because of poor positioning or spending too much time away from fights. The remaining 35% of unexplained variation, and the growing spread at high gold, show that gold alone does not determine output: two players with the same gold can deal very different amounts of damage.

The main strengths of the analysis are its large sample across three regions and four rank groups, the use of per-minute rates to control for game length, and the focus on one champion, which avoids comparing champions built for different roles.

There are also weaknesses. First, the relationship runs in both directions: gold buys items that raise damage, but damage also produces kills that pay gold. The slope therefore mixes both effects and should not be read as the damage that extra gold would cause. Second, the sample of players is not a simple random sample of all Jinx games. Players were drawn from rank ladders in equal numbers per rank group, so low-ranked games, which make up most real games, are under-represented relative to the player population. The slopes are similar across rank groups, which limits the effect of this imbalance. Third, the error variance is not constant, and the relationship has mild curvature, so predictions for games with very high gold are less precise than the residual standard error suggests. Fourth, the rank label comes from the sampled player, not the Jinx player.

Several extensions could address these issues. The match timelines already downloaded for this project record gold and damage minute by minute, so gold at 15 minutes could be used to predict later damage, reducing the feedback from damage to gold. Adding rank group as a predictor in a multiple regression would show whether the slope differs across skill levels. Repeating the analysis for other marksmen would show whether Jinx converts gold into damage more efficiently than similar champions.

References

Gelman, Andrew, Jennifer Hill, and Aki Vehtari. 2020. *Regression and Other Stories*. Cambridge: Cambridge University Press. https://doi.org/10.1017/9781139161879.

Kutner, Michael H., Christopher J. Nachtsheim, John Neter, and William Li. 2005. *Applied Linear Statistical Models*. 5th ed. Boston: McGraw-Hill/Irwin.

League of Graphs. 2026. "League of Graphs." Website. https://www.leagueofgraphs.com/.

Novak, Andrew R., Kyle J. M. Bennett, Matthew A. Pluss, and Job Fransen. 2020. "Performance Analysis in Esports: Modelling Performance at the 2018 League of Legends World Championship." *International Journal of Sports Science & Coaching* 15 (5-6): 809–17. https://doi.org/10.1177/1747954120932853.

R Core Team. 2024. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. https://www.R-project.org/.

Riot Games. 2009. "League of Legends." Video game. https://www.leagueoflegends.com/.

———. 2026a. "Jinx, the Loose Cannon." League of Legends champion page. https://www.leagueoflegends.com/en-us/champions/jinx/.

———. 2026b. "Match-V5 API." Riot Developer Portal. https://developer.riotgames.com/apis#match-v5.

———. 2026c. "Riot Developer Portal: General Policies." https://developer.riotgames.com/policies/general.

Wickham, Hadley, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Grolemund, et al. 2019. "Welcome to the tidyverse." *Journal of Open Source Software* 4 (43): 1686. https://doi.org/10.21105/joss.01686.